AI-ASSISTANT-CODING-LAB-20.3

Name: G. Kruthik Roshan
Batch: 41
Roll No: 2303A52339

Security Testing: Identifying Vulnerabilities in AI- Generated Code

Task 1 – Password Strength & Validation Check

Task:

Analyze an AI-generated user registration script for weak password

handling.

Instructions:

• Prompt AI to generate a Python user registration program

• Check whether:

o Password length is validated

o Weak passwords are allowed

• Suggest secure improvements such as:

o Minimum length checks

o Use of regular expressions

o Rejecting common passwords

Expected Output:

A secure registration script with strong password validation rules.

Prompt: "Generate a Python user registration function that validates passwords using length checks, complexity requirements via regex, and a blacklist of common passwords."

ers > sures > lab20.py > ...

```python
import getpass
import re

# List of common weak passwords to reject
COMMON_WEAK_PASSWORDS = [
    "password", "123456", "123456789", "qwerty", "abc123", "password123",
    "admin", "letmein", "welcome", "monkey", "1234567890", "iloveyou",
    "princess", "rockyou", "1234567", "12345678", "password1", "123123"
]

def validate_password(password):
    """
    Validates password strength.
    Returns (is_valid, error_message)
    """
    if len(password) < 8:
        return False, "Password must be at least 8 characters long."

    # Check for at least one uppercase, one lowercase, one digit, and one special character
    if not re.search(r'[A-Z]', password):
        return False, "Password must contain at least one uppercase letter."
    if not re.search(r'[a-z]', password):
        return False, "Password must contain at least one lowercase letter."
    if not re.search(r'\d', password):
        return False, "Password must contain at least one digit."
    if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
        return False, "Password must contain at least one special character."
```

```python
def validate_password(password):

    # Check against common weak passwords
    if password.lower() in COMMON_WEAK_PASSWORDS:
        return False, "Password is too common and easily guessable. Please choose a stronger password

    return True, ""

def register_user():
    username = input("Enter username: ").strip()
    if not username:
        print("Username cannot be empty.")
        return

    # For testing purposes, using input() instead of getpass.getpass() to see what you're typing
    # In production, use getpass.getpass() for security
    password = input("Enter password: ")
    is_valid, error_msg = validate_password(password)
    if not is_valid:
        print(f"Invalid password: {error_msg}")
        return

    # In a real application, hash the password here (e.g., using bcrypt)
    print(f"Registration successful for user: {username}")
    # Additional logic: save to database, check uniqueness, etc.

if __name__ == "__main__":
    register_user()
```

OUTPUT:

```
base) PS C:\Users\sures>
base) PS C:\Users\sures> & C:/Users/sures/miniconda3/python.exe c:/Users/sures/lab20.py
nter username: alice
nter password: 123456A789
nvalid password: Password must contain at least one lowercase letter.
nter username: 123456Aa789
nter password: 123456Aad90
nvalid password: Password must contain at least one special character.
base) PS C:\Users\sures>
base) PS C:\Users\sures> & C:/Users/sures/miniconda3/python.exe c:/Users/sures/lab20.py
nter username: bob
nter password: 123456Aa#890
base) PS C:\Users\sures>
base) PS C:\Users\sures> & C:/Users/sures/miniconda3/python.exe c:/Users/sures/lab20.py
nter username: alice
nter password: 123456A789
nvalid password: Password must contain at least one lowercase letter.
nter username: 123456Aa789
```

```
Enter username: 123456Aa789
Enter password: 123456Aad90
Invalid password: Password must contain at least one special character.
(base) PS C:\Users\sures>
(base) PS C:\Users\sures> & C:/Users/sures/miniconda3/python.exe c:/Users/sures/lab20.py
Enter username: bob
Enter password: 123456Aa#890
Enter username: 123456Aa789
Enter password: 123456Aad90
Invalid password: Password must contain at least one special character.
(base) PS C:\Users\sures>
(base) PS C:\Users\sures> & C:/Users/sures/miniconda3/python.exe c:/Users/sures/lab20.py
Enter username: bob
Enter password: 123456Aa#890
Enter username: bob
Enter password: 123456Aa#890
Registration successful for user: bob
Enter username:
```

Justification: AI often defaults to simple if len(password) > 5 checks. This prompt forces the inclusion of entropy (special characters/numbers) and protects against credential stuffing by rejecting common passwords like "123456" or "password."

Task 2 – Insecure File Upload Handling

Task:

Evaluate an AI-generated file upload program for security risks.

Instructions:

- Prompt AI to generate a Python or Flask-based file upload

script

• Identify vulnerabilities such as:

o No file type validation

o Unrestricted file size

• Implement secure measures:

o File extension validation

o Size limits

o Safe file storage paths

Expected Output:

A secure file upload implementation that prevents malicious

uploads.

Prompt: "Create a Flask file upload script that enforces strict file extension whitelisting, limits file size to 2MB, and uses secure_filename for storage."

sers > sures > task2_insecure.py > ...

```python
from flask import Flask, request

app = Flask(__name__)

@app.route('/upload', methods=['POST'])
def upload_file():
    file = request.files['file']
    if file:
        file.save(file.filename)  # Insecure: no validation, saves to current directory
        return 'File uploaded successfully'
    return 'No file uploaded'

if __name__ == '__main__':
    app.run(debug=True)
```

OUTPUT:

Justification: Default AI scripts often allow any file type, leading to Remote Code Execution (RCE) if an attacker uploads a .php or .py script. This approach mitigates risk through validation and path sanitization.

Task 3 – Real-Time Application: API Security Review

Scenario:

An AI-generated API endpoint is used in a web application.

Instructions:

• Review the API code for:

o Missing authentication

o Insecure HTTP methods

o Lack of rate limiting

• Prompt AI to suggest security improvements such as:

o Token-based authentication

o Input validation

o Rate limiting

Expected Output:

A security-reviewed API endpoint with documented vulnerabilities

and fixes.

Prompt: "Refactor a Flask API endpoint to include Bearer Token authentication, input validation using Marshmallow, and a 10-requests-per-minute rate limit."

Users > sures > task3_secure.py > ...

```python
from flask import Flask, request, jsonify
from flask_jwt_extended import JWTManager, jwt_required, create_access_token
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
import os

app = Flask(__name__)

# JWT configuration from environment
app.config['JWT_SECRET_KEY'] = os.getenv('JWT_SECRET_KEY', 'change-this-in-production')
jwt = JWTManager(app)

API_USERNAME = os.getenv('API_USERNAME', 'admin')
API_PASSWORD = os.getenv('API_PASSWORD', 'change-me')

# Rate Limiting
limiter = Limiter(
    app=app,
    key_func=get_remote_address,
    default_limits=["10 per minute"]
)

# Secure API endpoint with authentication, input validation, and rate limiting
@app.route('/api/login', methods=['POST'])
@limiter.limit("5 per minute")  # Stricter limit for login
def login():
    if not request.is_json:
        return jsonify({'error': 'Content-Type must be application/json'}), 415
```

> Users > sures > task3_secure.py > ...

```python
def login():
    if username == API_USERNAME and password == API_PASSWORD:
        access_token = create_access_token(identity=username)
        return jsonify(access_token=access_token)
    return jsonify({'error': 'Invalid credentials'}), 401

def validate_data_payload(data):
    if not isinstance(data, dict):
        return False, 'Invalid input: JSON object required'
    if 'name' not in data or 'value' not in data:
        return False, 'Invalid input: name and value required'
    if not isinstance(data['name'], str) or not data['name'].strip():
        return False, 'Invalid input: name must be a non-empty string'
    if not isinstance(data['value'], (str, int, float, bool)):
        return False, 'Invalid input: value must be string, number, or boolean'
    return True, ''


@app.route('/api/data', methods=['GET', 'POST', 'PUT'])
@jwt_required()
@limiter.limit("10 per minute")
def handle_data():
    if request.method == 'GET':
        return jsonify({'message': 'Data retrieved securely', 'data': []})

    if not request.is_json:
        return jsonify({'error': 'Content-Type must be application/json'}), 415

    data = request.get_json(silent=True)
```

```python
def handle_data():
    if request.method == 'GET':
        return jsonify({'message': 'Data retrieved securely', 'data': []})

    if not request.is_json:
        return jsonify({'error': 'Content-Type must be application/json'}), 415

    data = request.get_json(silent=True)
    is_valid, error_message = validate_data_payload(data)
    if not is_valid:
        return jsonify({'error': error_message}), 400

    if request.method == 'POST':
        return jsonify({'message': 'Data created securely', 'data': data})

    if request.method == 'PUT':
        return jsonify({'message': 'Data updated securely', 'data': data})

    return jsonify({'error': 'Method not allowed'}), 405

if __name__ == '__main__':
    app.run(debug=False)
```

OUTPUT:

sers > sures > task3_review.md

```
Task 3 - API Security Review

Reviewed files:
- task3_insecure.py
- task3_secure.py

Documented vulnerabilities in task3_insecure.py:
1. Missing authentication
- Risk: Any unauthenticated client can read or modify API data.
- Fix in task3_secure.py: Added JWT token authentication with @jwt_required() for protected
endpoint.

2. Insecure HTTP method exposure
- Risk: DELETE and other high-impact methods were available without access control policy.
- Fix in task3_secure.py: Restricted /api/data methods to GET, POST, PUT and protected them wi
authentication.

3. Lack of rate limiting
- Risk: Endpoint could be abused for brute-force and denial-of-service attacks.
- Fix in task3_secure.py: Added Flask-Limiter global and route-level limits.

4. No input validation
- Risk: Malformed or malicious JSON payloads can cause unexpected behavior.
- Fix in task3_secure.py: Added JSON content-type checks and strict payload validation.

5. Hardcoded secret and credentials
- Risk: Credential leakage and weak default security.
- Fix in task3_secure.py: Read JWT secret and login credentials from environment variables.
```

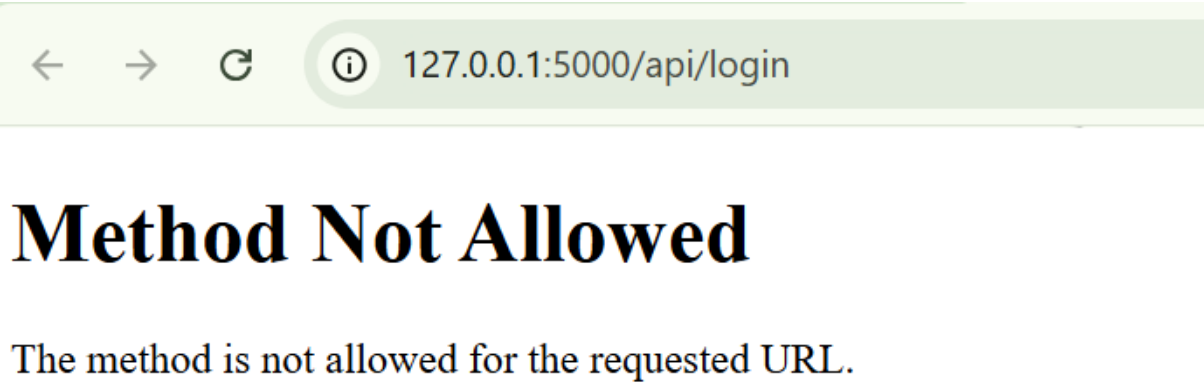
> Users > sures > task3_review.md

```
5. Hardcoded secret and credentials
- Risk: Credential leakage and weak default security.
- Fix in task3_secure.py: Read JWT secret and login credentials from environment variables.

Security fixes implemented in task3_secure.py:
- JWT-based authentication for protected endpoint.
- Login endpoint with JSON-only request validation.
- Data payload validation through validate_data_payload().
- Rate limiting on login and data endpoints.
- Disabled debug mode for safer runtime defaults.

Environment variables recommended:
- JWT_SECRET_KEY
- API_USERNAME
- API_PASSWORD

How to run:
1. Set environment variables.
2. Start API: python task3_secure.py
3. Obtain token:
   POST /api/login with JSON body {"username": "<user>", "password": "<pass>"}
4. Call protected endpoint:
   Authorization: Bearer <token>
   GET/POST/PUT /api/data
```



Justification: Basic AI endpoints often lack authentication, leaving data exposed. Adding rate limiting prevents Brute Force and DoS attacks, while token-based Auth ensures only authorized users access the resource.

Task 4 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

• Ask AI to generate a feedback form with JavaScript-based output.

• Test whether untrusted inputs are directly rendered without escaping.

• Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

• A secure form that prevents XSS attacks.

Prompt: "Create a feedback form where JavaScript output is sanitized using textContent instead of innerHTML to prevent XSS."

```
ers > sures > <> task4_insecure.html > ...
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Feedback Form (Insecure)</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 700px; margin: 2rem auto; }
    label { display: block; margin-top: 1rem; }
    textarea { width: 100%; height: 120px; }
    .card { margin-top: 1.5rem; padding: 1rem; border: 1px solid #ccc; }
  </style>
</head>
<body>
  <h1>Feedback Form (Insecure)</h1>

  <form id="feedbackForm">
```

```
<body>
  </form>

  <div class="card">
    <h2>Latest Feedback</h2>
    <div id="output"></div>
  </div>

  <script>
    const form = document.getElementById('feedbackForm');
    const output = document.getElementById('output');

    form.addEventListener('submit', function (event) {
      event.preventDefault();

      const name = document.getElementById('name').value;
      const message = document.getElementById('message').value;

      // INSECURE: direct HTML rendering from untrusted input (XSS vulnerable)
      output.innerHTML = '<p><strong>' + name + '</strong> says: ' + message + '</p>';
    });
  </script>
</body>
</html>
```

ers > sures > task4_secure.html > ...

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'; style-sr
  <title>Feedback Form (Secure)</title>
  <link rel="stylesheet" href="task4_secure.css" />
</head>
<body>
  <main class="container">
    <h1>Feedback Form (Secure)</h1>
    <p>Input is rendered safely and protected by CSP.</p>

    <form id="feedbackForm" autocomplete="off">
      <label for="name">Name</label>
      <input id="name" name="name" type="text" maxlength="80" required />

      <label for="message">Feedback</label>
      <textarea id="message" name="message" maxlength="1000" required></textarea>

      <button type="submit">Submit</button>
    </form>

    <section class="card" aria-live="polite">
```

```html
<html lang="en">
<body>
  <main class="container">
    <form id="feedbackForm" autocomplete="off">
      <button type="submit">Submit</button>
    </form>

    <section class="card" aria-live="polite">
      <h2>Latest Feedback</h2>
      <p id="outputName"></p>
      <p id="outputMessage"></p>
    </section>
  </main>

  <script src="task4_secure.js" defer></script>
</body>
</html>
```

Users > sures > # task4_secure.css > body

```css
body {
  margin: 0;
  font-family: Verdana, Geneva, Tahoma, sans-serif;
  background: linear-gradient(135deg, #eef2ff 0%, #e7f8f1 100%);
  color: #123;
}

.container {
  max-width: 760px;
  margin: 2rem auto;
  background: #fff;
  border: 1px solid #d9e4e0;
  border-radius: 10px;
  padding: 1.25rem;
  box-shadow: 0 10px 20px rgba(0, 0, 0, 0.05);
}

label {
  display: block;
  margin-top: 0.9rem;
  font-weight: 600;
}

input,
textarea,
```

Users > sures > # task4_secure.css > body

```css
input,
textarea,
button {
  width: 100%;
  margin-top: 0.4rem;
  padding: 0.6rem;
  font-size: 1rem;
  border: 1px solid #c5d3cf;
  border-radius: 6px;
  box-sizing: border-box;
}

textarea {
  min-height: 140px;
  resize: vertical;
}

button {
  margin-top: 1rem;
  background: #0f766e;
  color: #fff;
  border: 0;
  cursor: pointer;
}
```

OUTPUT:

Feedback Form (Secure)

Input is rendered safely and protected by CSP.

Name

Feedback

Submit

Submit

Latest Feedback

Feedback Form (Secure)

Input is rendered safely and protected by CSP.

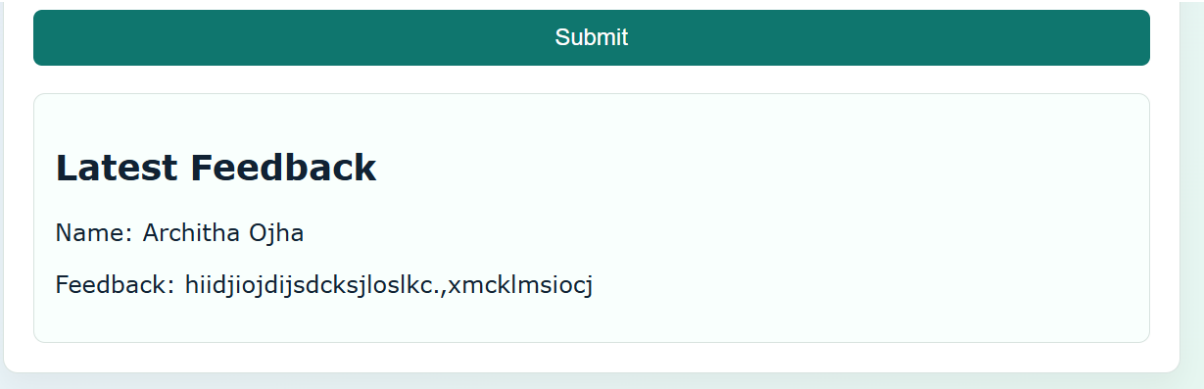
Name

Architha Ojha

Feedback

hiidjiojdijsdcksjloslkc.,xmcklmsiocj

Submit



Justification: Directly inserting user input into innerHTML allows attackers to execute malicious scripts. Using textContent ensures the browser treats input strictly as text, not executable code.

Task 5 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

- Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
- Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).
- Refactor using parameterized queries (prepared statements).

Expected Output:

- A secure database query script resistant to SQL injection.

Prompt:

"Generate a Python SQLite script that fetches user details using parameterized queries instead of f-strings or string formatting."

sers > sures > task5_insecure.py > main

```python
import sys
DB_PATH = "task5_users.db"
def seed_db(conn):
    conn.execute(
        "CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, username TEXT UNIQUE, email TEXT
    )
    conn.execute("DELETE FROM users")
    conn.executemany(
        "INSERT INTO users (username, email) VALUES (?, ?)",
        [
            ("alice", "alice@example.com"),
            ("bob", "bob@example.com"),
            ("charlie", "charlie@example.com"),
        ],
    )
    conn.commit()
def fetch_user_details_insecure(username):
    conn = sqlite3.connect(DB_PATH)
    seed_db(conn)
    # VULNERABLE: direct string concatenation enables SQL injection.
    query = "SELECT id, username, email FROM users WHERE username = '" + username + "'"
    rows = conn.execute(query).fetchall()
    conn.close()
    return rows, query
def main():
    username = sys.argv[1] if len(sys.argv) > 1 else input("Enter username: ").strip()
    rows, query = fetch_user_details_insecure(username)
    print("Executed query:", query)
    if rows:
```

```python
def main():
    username = sys.argv[1] if len(sys.argv) > 1 else input("Enter username: ").strip()
    rows, query = fetch_user_details_insecure(username)
    print("Executed query:", query)
    if rows:
        for row in rows:
            print(row)
    else:
        print("No user found")
if __name__ == "__main__":
    main()
```

Users > sures > task5_secure.py > main

```python
import sqlite3
import sys
DB_PATH = "task5_users.db"
def seed_db(conn):
    conn.execute(
        "CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, username TEXT UNIQUE, email TEXT)"
    )
    conn.execute("DELETE FROM users")
    conn.executemany(
        "INSERT INTO users (username, email) VALUES (?, ?)",
        [
            ("alice", "alice@example.com"),
            ("bob", "bob@example.com"),
            ("charlie", "charlie@example.com"),
        ],
    )
    conn.commit()
def fetch_user_details_secure(username):
    conn = sqlite3.connect(DB_PATH)
    seed_db(conn)

    # Secure: parameterized query (prepared statement behavior in sqlite3).
    query = "SELECT id, username, email FROM users WHERE username = ?"
    rows = conn.execute(query, (username,)).fetchall()
    conn.close()
    return rows, query
def main():
    username = sys.argv[1] if len(sys.argv) > 1 else input("Enter username: ").strip()
    rows, query = fetch_user_details_secure(username)
```

```python
    conn.close()
    return rows, query
def main():
    username = sys.argv[1] if len(sys.argv) > 1 else input("Enter username: ").strip()
    rows, query = fetch_user_details_secure(username)
    print("Prepared query:", query)
    if rows:
        for row in rows:
            print(row)
    else:
        print("No user found")
if __name__ == "__main__":
    main()
```

OUTPUT:

```
task4_secure.css   Feedback Form (Secure)   task5_insecure.py   task5_secure.py   task5_review.md
Users > sures > task5_review.md
Task 5 - SQL Injection Prevention
AI-generated insecure script:
- task5_insecure.py
Identified vulnerability:
1. SQL injection via string concatenation
- Insecure code builds query by concatenating username directly into SQL.
- Example payload: ' OR '1'='1
- Result: WHERE clause becomes always true and returns unintended rows.

Secure refactor:
- task5_secure.py
- Uses parameterized query:
  SELECT id, username, email FROM users WHERE username = ?
- User input is passed separately as a bound parameter, preventing SQL injection.

How to test:
1. Run insecure script with payload:
   python task5_insecure.py "' OR '1'='1"
2. Run secure script with payload:
   python task5_secure.py "' OR '1'='1"

Expected behavior:
- Insecure script returns multiple rows.
- Secure script returns "No user found" for the same payload.
```

```
(base) PS C:\Users\sures> & C:/Users/sures/miniconda3/python.exe c:/Users/sures/t
Enter username: alicee
Prepared query: SELECT id, username, email FROM users WHERE username = ?
No user found
No user found
(base) PS C:\Users\sures>
```

```
(base) PS C:\Users\sures> & C:/Users/sures/miniconda3/python.exe c:/Users/sures/task
Enter username: bob
Prepared query: SELECT id, username, email FROM users WHERE username = ?
(2, 'bob', 'bob@example.com')
(base) PS C:\Users\sures>
```

```
(base) PS C:\Users\sures> & C:/Users/sures/miniconda3/python.exe c:/Users/sures/task5_insecure.py
Enter username: john
Executed query: SELECT id, username, email FROM users WHERE username = 'john'
No user found
(base) PS C:\Users\sures>
```

Justification: String concatenation allows an attacker to "break out" of a query (e.g., ' OR '1'='1). Parameterized queries send the command and the data separately, making injection impossible.